# Software Requirements Specification

**for**

# Departmental Navigation System

**Version 1.0**

**Prepared by**

**GROUP NO. 1**

**Ajad ( *B230772CS* )**
**Amit Kumar ( *B230801CS* )**
**Rahul Chauhan ( *B231185CS* )**
**Simran Koshta ( *B230707CS* )**
**Utpal Gautam ( *B230598CS* )**

**Computer Science And Engineering Department**
**National Institute Of Technology Calicut**

**12/01/2026**

*Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.*

# Table of Contents

**Table of Contents** ii
**Revision History** ii
**1. Introduction** 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2
**2. Overall Description** 3
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
**3. External Interface Requirements** 7
3.1 User Interfaces 7
3.2 Hardware Interfaces 9
3.3 Software Interfaces 10
3.4 Communications Interfaces 11
**4. System Features** 12
4.1 User Authentication and Profile Management 12
4.2 Search and Location Discovery 13
4.3 Navigation and Route Finding 15
4.4 Offline Support and Caching 17
4.5 Administrative Dashboard and Content Management 18
4.6 Building and Floor Management 19
4.7 Accessibility Features 20
**5. Nonfunctional Requirements** 21
5.1 Performance Requirements 21
5.2 Safety Requirements 21
5.3 Security Requirements 22
5.4 Software Quality Attributes 22
5.5 Management Rules 23
**6. Other Requirements** 23
**Appendix A: Glossary** 24
**Appendix B: Analysis Models** 26
**Appendix C: To Be Determined List** 29

# Revision History

| Name | Date | Reason for Changes | Version |
|---|---|---|---|
| Initial Draft | January 12, 2026 | Project Requirements Specification | 1.0 |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) document specifies the functional and non-functional requirements for the Departmental Navigation System, a hybrid indoor-outdoor navigation application designed specifically for the Computer Science and Engineering (CSE) Department at the National Institute of Technology Calicut (NITC).

The system aims to address navigation challenges within the CSE department buildings by providing seamless navigation from outdoor campus locations to indoor department-specific destinations. This document covers all requirements for version 1.0 of the Departmental Navigation System, including mobile application requirements, backend infrastructure requirements, and administrative dashboard functionality.

## 1.2 Document Conventions

The following conventions are used throughout this SRS:

1. **Requirement IDs:** All functional requirements are identified with unique tags (e.g., FR-1.1, FR-2.1) and non-functional requirements use tags (e.g., NFR-1, NFR-2)
2. **Priority Levels:** Each requirement is assigned a priority of High (H), Medium (M), or Low (L)
3. **Emphasis:** Bold text indicates key terms, component names, and system functions
4. **Code/Technical Terms:** Monospace font (e.g., `Firebase Firestore`, `OSM API`) used for technical components
5. **Terminology:** Consistent terminology is maintained throughout; definitions provided in Glossary (Appendix A)
6. **Verifiability:** Requirements are stated as testable and measurable conditions where possible

## 1.3 Intended Audience and Reading Suggestions

This SRS is intended for multiple audiences:

- **Software Developers:** Detailed implementation requirements, technical specifications, and system architecture (Sections 3, 4, 5)
- **UI/UX Designers:** User interface requirements and user interaction patterns (Section 3.1)
- **System Architects:** System architecture, technology stack, and integration points (Section 3, Appendix B)
- **Department Administration:** Administrative features and data management (Section 4)

**Suggested Reading Sequence:**

- All readers: Start with Section 1 (Introduction) and Section 2 (Overall Description)
- Developers: Focus on Sections 3, 4, and Appendix B
- Designers: Review Section 3.1 (User Interfaces) in detail

## 1.4 Product Scope

The Departmental Navigation System is a mobile-first application that solves the critical problem of indoor navigation within the CSE department buildings at NITC. The system provides:

**Primary Objectives:**

- Accurate, turn-by-turn navigation from anywhere on campus to any location within CSE buildings
- Real-time location discovery through intelligent search functionality
- Seamless hybrid navigation combining outdoor (OSM API) and indoor (floor-based) routing
- Administrative tools for dynamic management of locations and facility information

**Business Goals:**

- Reduce time spent on navigation
- Minimize dependency on manual directions and third-party assistance
- Establish a scalable foundation for campus-wide smart navigation infrastructure

**System Scope:**

- **In Scope:** CSE department buildings navigation, faculty directory, laboratory locations, halls, administrative dashboard, offline indoor navigation support, user authentication and profile management
- **Out of Scope:** Other department buildings (future enhancement), AR/VR navigation (future enhancement), campus-wide expansion (future enhancement).

## 1.5 References

| Document | Version | Author | Date | Location |
|---|---|---|---|---|
| UI/UX Design Specifications | 1.0 | Design Team | January 2026 | Project Repository |
| Data Flow Diagrams | 1.0 | Architecture Team | January 2026 | Appendix B |
| API Documentation - OSM | Latest | Open Street Map (OSM) | N/A | https://taginfo.openstreetmap.in/taginfo/apidoc |
| Firebase Documentation | v9.0+ | Google Cloud | N/A | https://firebase.google.com/docs |
| Flutter Documentation | v3.0+ | Google Flutter | N/A | https://flutter.dev/docs |

# 2. Overall Description

## 2.1 Product Perspective

The Departmental Navigation System is a standalone mobile application designed to operate independently within NIT Calicut's CSE department while leveraging external geolocation and mapping services. It bridges the gap between outdoor campus navigation (via OSM/OpenStreetMap) and precise indoor localization.

**System Context:**

- **Primary Interface:** Mobile application (iOS and Android via Flutter)
- **Backend:** Cloud-based (Firebase infrastructure)
- **Third-Party Integration:** OSM API+GraphHopper+MapLibre for outdoor navigation
- **Data Storage:** Firestore NoSQL database with Cloud Storage for maps and assets

**Integration Points:**

- OSM API: Receives location queries, returns optimal outdoor routes
- Firebase Authentication: Manages user identity and access control
- Firestore Database: Stores and retrieves location, faculty, and administrative data
- Google Cloud Storage: Hosts floor maps, images, and application resources

## 2.2 Product Functions

The Departmental Navigation System provides the following major functional groupings:

**User-Facing Features:**

1. **Location Discovery & Search** - Find faculty cabins, labs, halls through intelligent search
2. **Navigation & Routing** - Get turn-by-turn directions from current location to destinations
3. **Hybrid Navigation** - Seamless outdoor-to-indoor navigation transitions
4. **Navigation Display -** Interactive 2D and 3D floor plan visualization with zoom, pan, and rotation capabilities
5. **Directory Management** - Access complete faculty and facility information
6. **User Authentication** - Secure registration, login, and profile management
7. **Offline Support** - Access cached maps and locations without internet connectivity

**Administrative Features:**

1. **Content Management** - Add, edit, delete, and update faculty information, cabin assignments, lab details, and hall information
2. **Map Management** - Upload, organize, and manage 2D and 3D floor plans and navigation maps
3. **User Management** - Monitor users, manage permissions, view activity logs
4. **System Analytics** - Track navigation patterns, popular destinations, usage statistics

### 2.3 User Classes and Characteristics

**Primary User Classes:**

**1. Regular Students (Frequency: Daily/Multiple times per day)**

- Technical Expertise: Moderate to High (CS students)
- Primary Goals: Quick navigation to labs, and faculty cabin
- Key Characteristics: Familiar with mobile apps, prefer quick interactions
- Requirements Subset: Search, navigation, offline access, history
- Priority: CRITICAL

**2. Faculty Members (Frequency: Regular, 2-3 times per day)**

- Technical Expertise: Moderate
- Primary Goals: Navigate to administrative offices, conference areas, departments
- Key Characteristics: Expect professional interface, value reliability
- Requirements Subset: Search, directory, navigation, profile management
- Priority: HIGH

**3. Visiting Scholars/Guests (Frequency: Occasional, during specific visits)**

- Technical Expertise: Variable (Low to High)
- Primary Goals: Find specific locations, understand campus layout
- Key Characteristics: First-time users, need intuitive interface, prefer detailed guidance
- Requirements Subset: Search, directory, turn-by-turn navigation, information
- Priority: HIGH

**4. System Administrators (Frequency: As-needed)**

- Technical Expertise: High
- Primary Goals: Maintain system infrastructure, manage security, configure APIs
- Key Characteristics: Require technical documentation, prefer command-line tools
- Requirements Subset: System configuration, API management, security settings
- Priority: MEDIUM

## 2.4 Operating Environment

## Hardware Requirements (Client):

| Component | Specification |
|---|---|
| Mobile Device | Android (v8.0 or later) or iOS (v12.0 or later) |
| Storage | Minimum 1GB available storage for offline maps and database |
| GPS Module | High-precision GPS receiver (±5-10 meters accuracy) |
| Motion Sensors | Accelerometer (for step detection in PDR) |
| Compass/Magnetometer | For heading and orientation sensing |

## Hardware Requirements (Server/Backend):

| Component | Specification |
|---|---|
| Cloud Platform | Google Cloud Platform (Firebase) |
| Database | Firestore (automatically scaled) |
| Hosting | Firebase Hosting (static assets) or Cloud Functions (backend logic) |

## Software Environment:

| Component | Specification |
|---|---|
| Mobile Framework | Flutter SDK v3.0 or later |
| Programming Language | Dart 3.0+ |
| IDE | Android Studio, VS Code, or IntelliJ IDEA |
| Version Control | Git (GitHub/GitLab) |

## External Services/APIs:

| Service | Purpose | Version |
|---|---|---|
| OSM API + GraphHopper + MapLibre | Outdoor routing and geocoding | Latest REST API |
| Firebase Firestore | NoSQL database | v1.0+ |
| Firebase Authentication | User authentication | v1.0+ |
| Firebase Cloud Storage | File storage and retrieval | v1.0+ |

## 2.5 Design and Implementation Constraints

## Mandatory Technical Constraints:

1. **Technology Stack Mandates:**
    - Mobile application MUST be developed using Flutter
    - Backend MUST use Firebase
    - Outdoor navigation MUST integrate OSM API + GraphHopper + MabLibre
2. **Platform Support:**
    - Application MUST support both Android (8.0+) and iOS (12.0+)
    - Minimum RAM requirement: 2GB (impacts UI responsiveness)
    - Must maintain consistent UX across different screen sizes (phone, tablet)
3. **Data Privacy & Compliance:**

- Adherence to data protection policies set by NITC administration
- User consent required for location tracking and device sensor access
- No storage of sensitive personal information beyond registration

4. **Security Mandates:**
   - Authentication MUST use Firebase Authentication (no custom auth systems)
   - Role-based access control (RBAC) mandatory for admin features
5. **Performance Standards:**
   - Application startup time: < 3 seconds
   - Search query response: < 2 seconds
   - Map rendering: < 1.5 seconds
   - Route calculation: < 3 seconds
   - Outdoor navigation start: < 2 seconds
6. **Offline Functionality:**
   - Indoor floor maps MUST be cached locally
   - Minimum offline functionality: View cached locations and maps

## 2.6 User Documentation

The following user documentation components will be delivered:

1. **In-app Help:** Context-sensitive help messages and tooltips for UI elements
2. **FAQ Section:** Common questions about navigation accuracy, offline maps, location search
3. **Administrator Manual (PDF):** Detailed documentation for administrative dashboard functions, data entry procedures, and access control configuration

## 2.7 Assumptions and Dependencies

**Assumptions:**

1. **User Environment Assumptions:**
   - Users will have internet connectivity for initial app download and data sync
   - Users own devices with minimum specifications
   - Users grant necessary permissions (location, storage)
2. **Technical Assumptions:**
   - OSM API will remain available and reliable during system operation
   - Device GPS accuracy sufficient for outdoor navigation (±5-10 meters acceptable)
3. **Organizational Assumptions:**
   - NITC administration will provide accurate faculty and facility information
   - Building access and layout information can be shared for in-app use

4. **Project Assumptions:**
    - Project will use Flutter as primary development framework
    - Firebase project already provisioned with required APIs enabled

**Dependencies:**

1. **External Service Dependencies:**
    - **OSM API Availability:** System cannot function without OSM API+GraphHopper+MapLibre
    - **Firebase Services:** Dependency on Firebase (Firestore, Authentication)
    - **Internet Connectivity:** Backend synchronization depends on user device connectivity
    - **GPS Signal:** Outdoor navigation dependent on GPS availability and accuracy
2. **Data Dependencies:**
    - Accurate building coordinates and entry points
    - Precise floor plans and layout information
    - Complete faculty directory with cabin locations
    - Lab inventory with descriptive information
3. **Organizational Dependencies:**
    - Department cooperation for data collection and validation
    - NITC IT support for API provisioning and configuration

---

# 3. External Interface Requirements

## 3.1 User Interfaces

The Departmental Navigation System provides two distinct user interfaces: the Mobile User Interface (for students, faculty, and visitors) and the Admin Dashboard (for administrative staff).

### 3.1.1 Mobile User Interface (Students/Visitors)

**Screen Structure:**

**Home Screen**

- Purpose: Primary entry point after login
- Components:
    - Search bar (prominent, top of screen) with placeholder "Search faculty, labs, halls"
    - Current location display (if GPS enabled)
    - Recent searches (quick access)
    - Navigation drawer (hamburger menu)
    - Bottom navigation bar (Home, Directory, Maps, Profile)

**Search Results Screen**

- Purpose: Display search results with location details
- Components:
    - Search bar with filters (Category: Faculty/Lab/Hall)
    - Each result card shows: name, location, floor number, contact info
    - "Start Navigation" button for each result

**Navigation Screen**

- Purpose: Provide turn-by-turn directions
- Components:
    - Interactive map (50% of screen)
    - Turn-by-turn instructions list
    - Distance remaining and estimated time
    - Current step highlighted
    - Start/Stop navigation buttons

**Faculty Directory Screen**

- Purpose: Browse all faculty with contact information
- Components:
    - Searchable list of faculty
    - Faculty card showing: name, designation, cabin location, email
    - Quick "Navigate To Cabin" button

**Building/Floor Maps Screen**

- Purpose: Interactive floor plan viewing
- Components:
    - Zoomable, pannable floor plan image
    - Location pins with labels
    - Floor selector (dropdown or carousel)
    - Building selector dropdown
    - Search highlights on map

**User Profile Screen**

- Purpose: Manage user account and preferences
- Components:
    - User name and email display
    - Account settings (change password, email preferences)
    - Search history
    - Logout button

### 3.1.2 Admin Dashboard

**Screen Structure:**

**Dashboard Home**

- Purpose: System overview and quick actions
- Components:
    - Key metrics (total users, searches today, popular locations)
    - Quick action buttons (Add Location, Upload Map, User Management)

**Content Management**

- Faculty Management: List view with add/edit/delete
- Laboratory Management: Lab details, contact info, equipment list
- Hall Management: facilities, accessibility features
- Location Management: Buildings, floors, entry points, coordinate management

**Map Management**

- Floor plan upload interface
- Floor assignment management
- Map preview and validation

**User Management**

- User list with sorting/filtering
- User role assignment (Student/Visitor/Admin)
- Account status management (active/inactive/suspended)
- Permission settings
- Activity logs per user

**Analytics Dashboard**

- Navigation usage statistics (searches, routes)
- Popular locations heatmap
- Peak usage times
- System performance metrics
- Export reports (PDF)

**System Settings**

- API configuration (OSM API key management)
- Maintenance windows
- Backup/restore options

## 3.2 Hardware Interfaces

The system interfaces with the following hardware components:

## Mobile Device Components (Client-Side):

| Hardware | Purpose | Requirements |
|---|---|---|
| GPS Receiver | Location tracking | Supports GPS |

| | | |
|---|---|---|
| Accelerometer | Pedestrian dead reckoning | Detects movement direction and distance |
| Magnetometer | Compass functionality | Provides magnetic heading for directions |
| Wi-Fi/Cellular Modem | Network connectivity | For API calls and data synchronization |
| Storage | Local data persistence | For offline maps and location cache |

**Sensor Data Flow:**

GPS → Location Update (1 per second)
↓
Accelerometer → Movement Tracking (10-50Hz)
↓
Magnetometer → Heading Calculation (1 per second)
↓
Application Logic → Pedestrian Dead Reckoning (PDR) Position Calculation
↓
UI Rendering → Map Update with Current Location

## 3.3 Software Interfaces

### 3.3.1 Third-Party Service Integrations:

**OSM + GrapHopper + MapLibre**

- **Purpose:** Outdoor routing, geocoding, place search
- **Integration Method:** REST API calls from Flutter client
- **Data Exchange:** JSON request/response format
- **Error Handling:** Specific error codes for quota exceeded, invalid request, etc.

**Firebase Firestore Database**

- **Purpose:** Store and retrieve building, location, faculty, and user data
- **Collections:**
  - **Collection:** faculty (documents: faculty IDs with name, email, phone, cabin location)
  - **Collection:** locations (documents: location IDs with name, building, floor, coordinates, type)
  - **Collection:** buildings (documents: building info with entry points, address)
  - **Collection:** floor_maps (documents: map metadata, image URLs, scale information)
- **Access Control:** Role-based security rules enforced at database level

**Firebase Authentication**

- **Purpose:** User identity management and access control
- **Methods Supported:**
  - Email/password authentication
  - Google Sign-In (optional)

- **Token Management:** Firebase JWT tokens with 1-hour expiry
- **Session Management:** Automatic refresh tokens for persistent login

**Firebase Cloud Storage**

- **Purpose:** Store floor maps, images, and media files
- **Access Control:** Firestore security rules control read/write permissions

### 3.3.2 Internal Component Interfaces:

**Navigation Engine ↔ Location Database**

- Input: User's current location, destination coordinates
- Output: Sequence of navigation steps (turn-by-turn)
- Processing: Route calculation, floor transition detection, step generation

**Search Engine ↔ Firestore Database**

- Input: Search query string
- Output: Matching locations with distance and metadata
- Processing: Full-text search, fuzzy matching, result ranking

**Offline Cache ↔ Firestore Sync**

- Input: Newly cached data, sync interval
- Output: Updated local database with latest information
- Processing: Incremental sync, conflict resolution

## 3.4 Communications Interfaces

**Network Communication:**

| Protocol | Purpose | Security |
|---|---|---|
| HTTPS | All API communications | Encryption in transit |
| Firebase Realtime | Real-time location updates (admin→users) | Firebase security rules |
| REST API | Standard request/response | OAuth 2.0 token validation |

**Communication Standards:**

- **API Response Time:** < 2 seconds for standard queries
- **Data Synchronization:** Automatic every 30 minutes or manual trigger
- **Timeout Values:** 30 seconds for API calls, 60 seconds for large uploads

# 4. System Features

## 4.1 FR1: User Authentication and Profile Management

### 4.1.1 Description and Priority

**Priority:** HIGH

**Overview:** This feature manages user registration, login, profile management, and role-based access control. It ensures secure user identification and controls access to user-specific and admin features.

### 4.1.2 Stimulus/Response Sequences

**Sequence 1: User Registration**

1. User opens app and taps "Sign Up", registration form opens
2. User enters credentials and taps "Register"
3. System validates email format and password strength and create account
4. System displays "Registration successful" and navigates to home screen

**Sequence 2: User Login**

1. User enters email and password
2. System authenticates against Firebase
3. If authentication successful: System loads user preferences and navigates to home screen
4. If authentication failed: System displays "Invalid credentials" error message
5. User can tap "Forgot Password" to reset

**Sequence 3: Profile Update**

1. User navigates to Profile Screen
2. User edits name, preferences, or settings
3. User taps "Save Changes"
4. System validates changes and updates Firestore
5. System displays "Profile updated successfully"

### 4.1.3 Functional Requirements

**FR-1.1: User Registration**

- System SHALL provide email/password registration mechanism
- System SHALL enforce password requirements: minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 number, 1 special character
- System SHALL prevent duplicate email addresses (case-insensitive)
- User account SHALL be automatically assigned "Student" role on creation
- System SHALL store password using Firebase Authentication (bcrypt hashing, no plaintext storage)
- **Acceptance Criteria:** User can successfully register with valid email/password; registration email received within 30 seconds.

**FR-1.2: User Login**

- System SHALL authenticate users against Firebase Authentication service
- System SHALL display error message for invalid credentials (without revealing which field is invalid)
- System SHALL provide "Forgot Password" functionality
- System SHALL restrict access to unauthenticated users (redirect to login)
- **Acceptance Criteria:** Valid credentials allow login within 2 seconds; invalid credentials rejected; session persists across app restarts

**FR-1.3: Password Management**

- System SHALL send password reset email with secure token
- System SHALL allow users to change password from profile screen
- System SHALL require current password verification before password change
- **Acceptance Criteria:** Password reset email received within 30 seconds; reset completes within 2 minutes; new password takes effect immediately

**FR-1.4: Profile Management**

- System SHALL allow users to view and edit profile information
- System SHALL allow users to set navigation preferences (walking speed)
- System SHALL allow users to view search history
- **Acceptance Criteria:** All profile fields editable and persist after logout; preferences applied within 1 second of change

**FR-1.5: Role-Based Access Control**

- System SHALL support three user roles: Student, Visitor, Admin
- System SHALL automatically assign "Student" role to new registered users
- System SHALL restrict admin features to Admin role only
- **Acceptance Criteria:** Different UI displayed based on role; non-admin users cannot access admin features; role changes take effect immediately

**FR-1.6: Multi-Session Management**

- System SHALL support logout functionality
- System SHALL clear all sensitive data on logout
- System SHALL require re-authentication if backgrounded for more than 7 days
- **Acceptance Criteria:** Logout successful; session properly cleared; backgrounded sessions handled correctly

## 4.2 FR2: Search and Location Discovery

### 4.2.1 Description and Priority

### **Priority:** HIGH

**Overview:** This feature enables users to discover and search for locations within the CSE department through intelligent search functionality, autocomplete suggestions, filtering options, and browsable directory.

### 4.2.2 Stimulus/Response Sequences

**Sequence 1: Basic Search**

1. User taps search bar on home screen
2. System displays autocomplete suggestions in real-time
3. System displays detailed result with location information
4. User can tap "Navigate" button to start navigation

**Sequence 2: Filtered Search**

1. User opens search screen
2. User enters search query
3. System displays results with filter options
4. User selects filters (Category: Faculty/Lab/Hall)
5. User selects a result

**Sequence 3: Directory Browsing**

1. User navigates to Directory tab
2. System displays categorized list (Faculty, Laboratories, Lecture Halls)
3. System displays all faculty with names, designations, cabin locations
4. System shows detailed card with contact information and map

### 4.2.3 Functional Requirements

**FR-2.1: Global Location Search**

- System SHALL provide search functionality accessible from multiple screens
- System SHALL search across faculty names, lab names, hall designations
- System SHALL show distance to result if user's current location is available
- **Acceptance Criteria:** Query "Sharma" returns all faculty with surname Sharma; "Lab" returns all laboratories; results displayed within 2 seconds

**FR-2.2: Auto-Complete Suggestions**

- System SHALL display autocomplete suggestions as user types
- System SHALL prioritize recently searched items and saved locations
- **Acceptance Criteria:** Suggestions appear within 500ms of typing

**FR-2.3: Search History**

- System SHALL store last 10 search queries per user
- System SHALL display search history in search suggestions
- System SHALL allow clearing individual search history items
- System SHALL persist search history across app restarts
- **Acceptance Criteria:** History persists across sessions; user can clear history; deleted items not recovered

**FR-2.4: Faculty Directory**

- System SHALL display complete list of faculty members
- System SHALL show for each faculty: name, designation, cabin location, email

- System SHALL allow "Navigate to Cabin" action from directory
- System SHALL be searchable within directory
- **Acceptance Criteria:** All faculty listed with complete information; search within directory works; navigation action functions correctly

**FR-2.5: Laboratory Directory**

- System SHALL display complete list of laboratories
- System SHALL show for each lab: name, room number, floor
- System SHALL allow "Navigate to Lab" action
- **Acceptance Criteria:** All labs listed with details, navigation initiates properly

**FR-2.6: Hall Directory**

- System SHALL display complete list of halls
- System SHALL allow "Navigate to Hall" action
- **Acceptance Criteria:** All halls listed, navigation works

**FR-2.7: Location Search Filters**

7. System SHALL provide filters: Category (Faculty/Lab/Hall)
8. System SHALL update search results in real-time as filters change
9. System SHALL clear filters option available
10. System SHALL default to "All" for each filter when search initiates
11. **Acceptance Criteria:** Filters reduce result set correctly; clearing filters resets to all results

**FR-2.8: Search Result Display**

- System SHALL display each result as a card with icon, name, location, distance
- System SHALL show building and floor information for each result
- System SHALL provide "Navigate" buttons per result
- System SHALL highlight matching search term in results
- **Acceptance Criteria:** All result information displayed clearly; buttons functional; highlighting visible

## 4.3 FR3: Navigation and Route Finding

### 4.3.1 Description and Priority

**Priority:** CRITICAL

**Overview:** This feature provides turn-by-turn navigation guidance combining outdoor navigation (using OSM API) with indoor floor-based navigation. It supports hybrid transitions from outdoor to indoor and provides real-time location tracking.

### 4.3.2 Stimulus/Response Sequences

**Sequence 1: Start Outdoor Navigation**

1. User selects destination location
2. System retrieves current location from GPS

3. System calls OSM Directions API
4. System displays route on map with distance
5. System initiates turn-by-turn guidance
6. As user moves, system updates current position on map

**Sequence 2: Transition from Outdoor to Indoor**

1. User approaches building entry point
2. System detects proximity to entry point
3. System displays indoor floor map with route overlay
4. System displays step-by-step indoor navigation

**Sequence 3: Indoor Navigation**

1. System displays floor plan of current floor
2. System shows location pin for destination
3. System displays turn-by-turn directions
4. As user moves, system updates position on floor map (if PDR enabled)
5. System displays distance remaining
6. Upon arrival, system displays "Destination reached"

### 4.3.3 Functional Requirements

**FR-3.1: Outdoor Navigation (OSM + GraphHopper + MabLibre Integration)**

- System SHALL integrate OSM Directions API
- System SHALL retrieve user's current location using device GPS
- System SHALL calculate optimal route from current location to building entry
- System SHALL display route on interactive map
- System SHALL provide turn-by-turn visual directions
- System SHALL update route in real-time as user's location changes
- **Acceptance Criteria:** Route calculated within 3 seconds; turns displayed correctly

**FR-3.2: Building Entry Point Detection**

- System SHALL store entry point coordinates for CSE buildings
- System SHALL detect when user is within the building entry point (geofencing)
- System SHALL display building name,image (entry) when entry point approached
- System SHALL automatically switch
- **Acceptance Criteria:** Entry points correctly detected; no false triggers

**FR-3.3: Indoor Floor Navigation**

- System SHALL display floor plan
- System SHALL show user's selected destination as pin on floor plan
- System SHALL display route on floor plan with clear path indication
- System SHALL be zoomable and pannable (pinch zoom, drag)

- System SHALL show building name with floor selector dropdown
- **Acceptance Criteria:** Floor maps load within 1.5 seconds; zoom/pan smooth; destination clearly visible

**FR-3.4: Turn-by-Turn Indoor Directions**

- System SHALL generate step-by-step directions from current floor to destination
- System SHALL show distance for each step
- **Acceptance Criteria:** Directions clear and sequential; steps easy to follow

**FR-3.5: Floor Transition Navigation**

- System SHALL show current and target floor numbers
- System SHALL display on map: location of stairs/elevators to use
- System SHALL resume navigation after floor change
- **Acceptance Criteria:** Floor transitions clear; instructions for reaching stairs/elevators provided; navigation resumes correctly

**FR-3.6: Pedestrian Dead Reckoning (Optional)**

- System MAY implement PDR for real-time indoor positioning
- If implemented: System SHALL use accelerometer for movement tracking
- System SHALL display user's estimated position on indoor floor map
- System SHALL periodically recalibrate
- **Acceptance Criteria:** Position estimate within meters of actual position; PDR works without GPS

**FR-3.7: Navigation History and Favorites**

- System SHALL store navigation history (last 10 routes)
- System SHALL allow saving frequently visited locations as favorites
- System SHALL display favorites in quick access on home screen
- System SHALL allow deleting navigation history items
- **Acceptance Criteria:** Favorites saved and retrievable; quick access functional; history persists across sessions

**FR-3.8: Navigation Settings**

- System SHALL allow users to set walking speed preference (slow, normal, fast)
- System SHALL allow enabling/disabling location history
- **Acceptance Criteria:** Settings persist across sessions

## 4.4 FR4: Offline Support and Caching

### 4.4.1 Description and Priority

**Priority:** HIGH

**Overview:** This feature enables users to access indoor floor maps and location information without internet connectivity after initial data synchronization.

### 4.4.2 Functional Requirements

**FR-4.1: Floor Map Caching**

- System SHALL cache floor maps locally after first download
- System SHALL allow users to manually download maps for offline access
- System SHALL display data storage size of cached maps
- System SHALL update cached maps automatically on app launch if internet available
- **Acceptance Criteria:** Floor maps accessible without internet; storage size displayed; user can manage cached data

**FR-4.2: Location Information Caching**

- System SHALL cache faculty, laboratory, and hall information locally
- **Acceptance Criteria:** Location data available offline; sync works reliably; last sync time displayed

**FR-4.3: Offline Navigation**

- System SHALL support indoor navigation using cached floor maps
- System SHALL allow searching cached locations offline
- System SHALL display "Offline Mode" indicator when not connected
- **Acceptance Criteria:** Indoor navigation works offline; offline mode clearly indicated; search functions with cached data

## 4.5 FR5: Admin Dashboard and Content Management

### 4.5.1 Description and Priority

**Priority:** HIGH

**Overview:** This feature provide administrative interface for managing buildings, locations, floor maps, users, and system analytics.

### 4.5.2 Functional Requirements

**FR-5.1: Location Management (CRUD)**

- System SHALL provide admin interface to create new locations
- System SHALL allow editing existing location details
- System SHALL allow deleting locations
- System SHALL display location list with sorting by name, type, floor, building
- System SHALL validate location coordinates before saving
- System SHALL prevent deletion of locations with active navigation history
- **Acceptance Criteria:** All CRUD operations functional; validation prevents invalid data; bulk import supports 500+ records; audit logs maintained

**FR-5.2: Floor Map Management**

- System SHALL provide interface to upload floor plan images (PNG, JPG, PDF)
- System SHALL allow setting map boundaries/calibration points
- System SHALL display map preview before publishing

- **Acceptance Criteria:** Map upload successful within 30 seconds; versioning works; preview displays correctly; users notified of updates

**FR-5.3: Faculty Information Management**

- System SHALL allow adding faculty members with details
- System SHALL support editing faculty information
- System SHALL link faculty to cabin locations automatically
- **Acceptance Criteria:** Faculty data CRUD functional; email validation works; profile pictures upload correctly

**FR-5.4: User Management**

- System SHALL display list of registered users with email, registration date, last login, role
- System SHALL allow filtering users by role, registration date, activity status
- System SHALL allow deactivating user accounts (not deletion, preserves data)
- System SHALL allow resetting user passwords
- **Acceptance Criteria:** User list complete and sortable; deactivation prevents login; export functional

**FR-5.5: Analytics and Reporting**

- System SHALL display dashboard with key metrics (total users, searches today, popular locations)
- System SHALL show usage trends (daily/weekly/monthly searches and navigations)
- System SHALL display heatmap of popular locations
- System SHALL show peak usage times
- **Acceptance Criteria:** All metrics load within 5 seconds; reports accurate and downloadable; heatmap updates daily; trends reflect actual usage

**FR-5.6: System Configuration**

- System SHALL provide settings for OSM API key management
- System SHALL allow setting maintenance windows
- **Acceptance Criteria:** Configuration changes take effect immediately; backup/restore successful; system status accurate

## 4.6 FR6: Building and Floor Management

### 4.6.1 Description and Priority

**Priority:** HIGH

**Overview:** This feature manages building information, floor structures, entry points, and relationships between buildings and locations.

### 4.6.2 Functional Requirements

**FR-6.1: Building Information Management**

- System SHALL store building details: name, coordinates (lat/long), address
- System SHALL allow adding/editing/deleting buildings
- System SHALL validate building coordinates (within NITC campus)

- System SHALL display list of buildings
- System SHALL link buildings to multiple floors
- System SHALL store entry point coordinates for each building
- **Acceptance Criteria:** Building CRUD functional; coordinates validated; entry points correctly stored

**FR-6.2: Floor Structure Management**

- System SHALL store floor information for each building: floor number
- System SHALL link floor maps to floor records
- System SHALL allow ordering floors
- **Acceptance Criteria:** Floors display in correct order; floor maps linked properly

**FR-6.3: Entry Point Management**

- System SHALL store building entry point coordinates (latitude, longitude)
- System SHALL allow multiple entry points per building
- System SHALL guide users to nearest entry point based on current location
- System SHALL allow editing entry point information
- **Acceptance Criteria:** Entry points correctly identified; navigation to nearest entry point works; multiple entries supported

**FR-6.4: Location to Building/Floor Mapping**

- System SHALL link all locations (faculty cabins, labs, halls) to specific building and floor
- System SHALL validate floor assignment (floor exists in building)
- System SHALL maintain location hierarchy (Building → Floor → Location)
- **Acceptance Criteria:** All locations properly mapped; hierarchy maintained; re-assignments update successfully

## 4.7 FR7: Accessibility Features

### 4.7.1 Description and Priority

**Priority:** MEDIUM

**Overview:** This feature ensures the application is accessible to users with different abilities.

### 4.7.2 Functional Requirements

**FR-7.1: Visual Accessibility**

- System SHALL support high contrast mode
- System SHALL support system dark mode preference
- **Acceptance Criteria:** High contrast mode functional; contrast verified with tools

# 5. Nonfunctional Requirements

## 5.1 Performance Requirements

**5.1.1 Response Time Requirements**

| Operation | Response Time Requirement | Acceptance Criteria |
|---|---|---|
| App Startup | < 3 seconds | Splash screen → Home screen |
| Search Query | < 2 seconds | Query entered → Results displayed |
| Route Calculation | < 3 seconds | Destination selected → Route shown |
| Map Rendering | < 1.5 seconds | Floor selected → Map displayed |
| Navigation Start | < 2 seconds | Route selected → Turn-by-turn begins |
| Location Sync | < 30 seconds | Sync triggered → Complete |
| Floor Plan Load (offline) | < 1 second | From local cache |
| API Response (average) | < 500ms | Network dependent |

#### 5.1.2 Throughput Requirements

- System SHALL support simultaneous navigation by 100+ users without performance degradation
- System SHALL process 100+ concurrent Firestore reads/writes
- System SHALL cache locally to reduce server load during peak hours

#### 5.1.3 Scalability Requirements

- Database SHALL automatically scale to support 2x current user load
- System SHALL handle 10x peak load without service interruption
- System SHALL be designed to support campus-wide expansion

### 5.2 Safety Requirements

#### 5.2.1 Location Privacy and Safety

- System SHALL NOT track user location without explicit permission
- System SHALL display clear indication when location tracking is active
- System SHALL warn users before accessing sensitive permissions

#### 5.2.2 Navigation Safety

- System SHALL warn users about stairs, elevators
- System SHALL recommend safe routes
- System SHALL provide clear instructions to avoid user disorientation

### 5.3 Security Requirements

#### 5.3.1 Authentication and Authorization

- System SHALL enforce role-based access control (RBAC)

- System SHALL NOT allow unauthorized access to admin features
- System SHALL not store passwords in plaintext
- System SHALL enforce password change every 90 days for admin accounts
- System SHALL lock account after 5 failed login attempts for 30 minutes
- **Acceptance Criteria:** Unauthorized access attempts blocked; passwords hashed; account lockout enforced; logs maintained

### 5.3.2 Data Protection

- System SHALL encrypt sensitive data at rest (Firebase encryption)
- System SHALL sanitize all user input to prevent SQL injection and XSS attacks
- System SHALL validate all API requests for data type, range, and format
- **Acceptance Criteria:** TLS implemented; input validation working; rate limiting enforced; backups completed daily

### 5.3.3 Data Privacy Compliance

- System SHALL comply with NITC data privacy policies
- System SHALL maintain privacy policy document (accessible in-app)
- System SHALL display data usage terms clearly during registration
- System SHALL not use personal data for purposes other than stated
- **Acceptance Criteria:** Privacy policy available; data export works; account deletion removes all data

## 5.4 Software Quality Attributes

### 5.4.1 Reliability

- System SHALL not lose user data due to app crashes
- System SHALL validate all critical transactions (user registration, location updates)
- **Acceptance Criteria:** Uptime monitored and reported; automatic recovery works; crash reporting functional; data integrity maintained

### 5.4.2 Maintainability

- Code SHALL follow Dart style guide and best practices
- Code SHALL have meaningful variable and function names
- Code SHALL support easy debugging (logging, error messages)
- **Acceptance Criteria:** Code review passes; test coverage meets minimum; documentation complete

### 5.4.3 Portability

- System SHALL run on Android 8.0+ and iOS 12.0+
- System SHALL support both portrait and landscape orientations
- System SHALL work on devices with 2GB to 8GB RAM
- **Acceptance Criteria:** Android and iOS; orientations functional; responsive on all sizes

### 5.4.4 Usability

- System SHALL have intuitive navigation requiring minimal training
- System SHALL provide in-app help and tooltips for new users
- System SHALL use familiar UI patterns (Material Design 3, Human Interface Guidelines)
- **Acceptance Criteria:** Usability testing shows 95%+ task completion; help system functional; familiar patterns used

## 5.5 Management Rules

### 5.5.1 Access Control Rules

- RULE BR-1: Only Admin role can access admin dashboard
- RULE BR-2: Users cannot modify other users' saved locations or history

### 5.5.2 Data Management Rules

- RULE BR-3: Faculty cabin information must be verified
- RULE BR-4: Search history retained for 6 months, then auto-deleted

### 5.5.3 System Operation Rules

- RULE BR-5: User accounts inactive for 1 year will be flagged for review
- RULE BR-6: Failed login attempts more than 5 times trigger temporary account lockout
- RULE BR-7: Admin password must be changed every 90 days

---

# 6. Other Requirements

## 6.1 Database Requirements

### 6.1.1 Firestore Database Schema

**Collection: buildings**

- Field: buildingId (unique identifier)
- Field: name (string, required)
- Field: coordinates (geopoint, required)
- Field: entryPoints (array of geopoints)
- Field: description (string)

**Subcollection: buildings/{buildingId}/floors**

- Field: floorId (unique identifier)
- Field: floorNumber (integer)
- Field: mapImageUrl (string, storage reference)
- Field: mapBoundaries (object with coordinates)

**Subcollection: buildings/{buildingId}/floors/{floorId}/locations**

- Field: locationId (unique identifier)
- Field: type (enum: faculty, lab, hall)

- Field: name (string)
- Field: description (string)
- Field: coordinates (geopoint)
- Field: contactEmail (string)
- Field: facilities (array of strings)

**Collection: users**

- Field: userId (UID from Firebase Auth)
- Field: email (string, unique, required)
- Field: displayName (string)
- Field: role (enum: student, visitor, admin)
- Field: preferences (object)
- Field: lastLogin (timestamp)

**Subcollection: users/{userId}/searchHistory**

- Field: searchId (unique identifier)
- Field: query (string)
- Field: resultCount (number)
- Field: timestamp (timestamp)
- Field: resultClicked (locationId, optional)

---

# Appendix A: Glossary

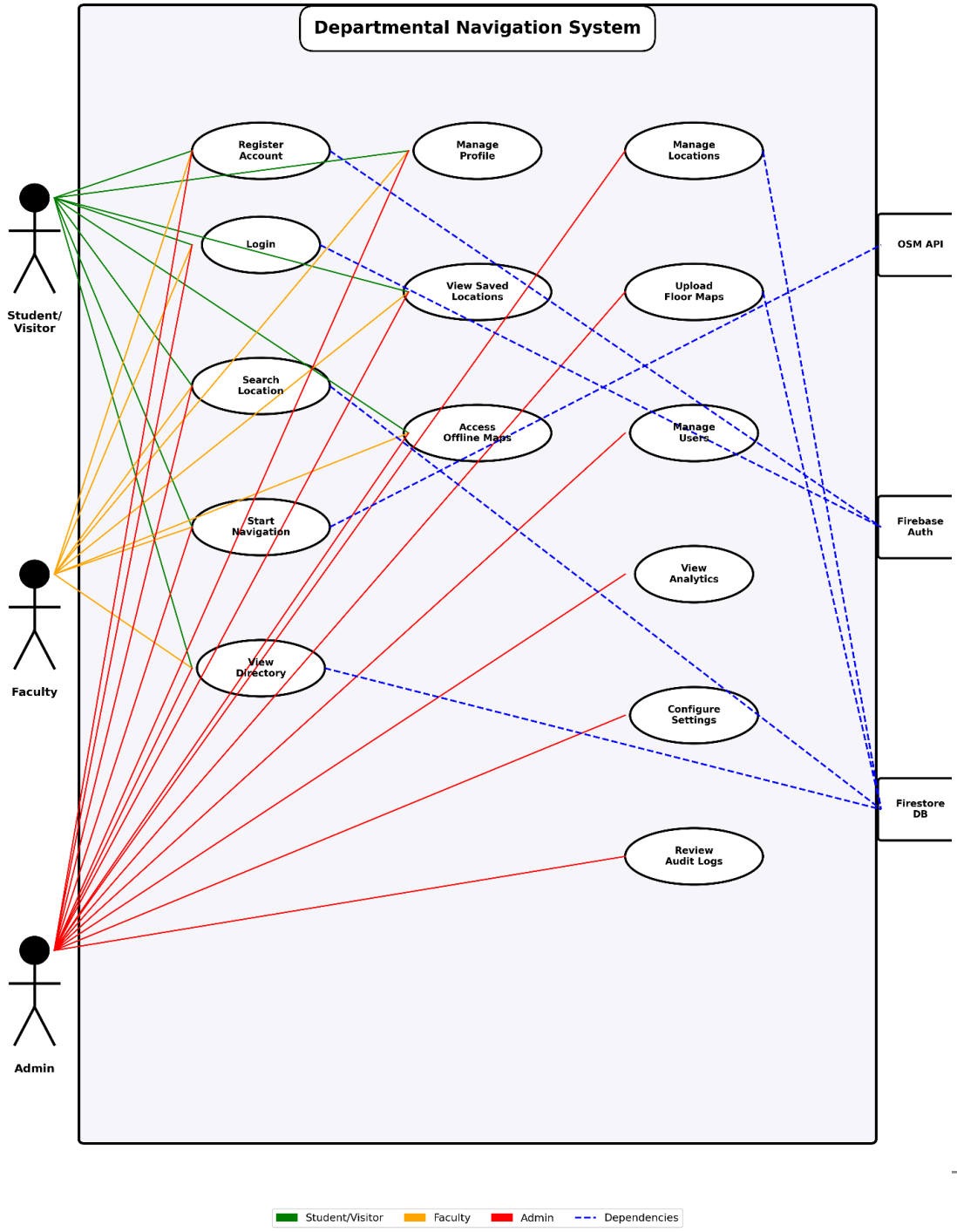
| Term | Definition |
|---|---|
| **API** | Application Programming Interface; interface for communication between software systems |
| **Building Entry Point** | Designated entrance to a building where outdoor navigation transitions to indoor |
| **Firebase** | Google's cloud platform providing authentication, database, storage, and hosting services |
| **Firestore** | NoSQL cloud database provided by Firebase |
| **GeoFencing** | Defining virtual geographic boundaries to trigger location-based events |
| **GPS** | Global Positioning System; satellite-based positioning system for outdoor location |
| **GUI** | Graphical User Interface; visual interface for user interaction |

| | |
|---|---|
| **JWT** | JSON Web Token; secure token format for authentication |
| **PDR** | Pedestrian Dead Reckoning; position estimation using motion sensors |
| **Role-Based Access Control (RBAC)** | Security model restricting access based on user role |
| **SRS** | Software Requirements Specification; document defining system requirements |
| **TLS** | Transport Layer Security; encryption protocol for secure communication |
| **UI/UX** | User Interface and User Experience; design of user-facing systems |
| **User Authentication** | Process of verifying user identity (login) |
| **User Authorization** | Process of determining user permissions after authentication |
| **CSE Department** | Computer Science and Engineering Department at NITC |
| **NITC** | National Institute of Technology Calicut; the institution deploying this system |
| **Faculty** | Academic staff members (professors, assistant professors) |
| **Lecture Hall** | Classroom designed for large-scale teaching |
| **Laboratory** | Dedicated space for practical learning and research |
| **Floor Map** | 2D or 3D representation of building floor layout |
| **Smart Campus** | Campus infrastructure utilizing IoT, mobile, and data analytics |
| **Navigation** | Process of determining and following route from source to destination |
| **Hybrid Navigation** | Combination of outdoor (GPS-based) and indoor (map-based) navigation |

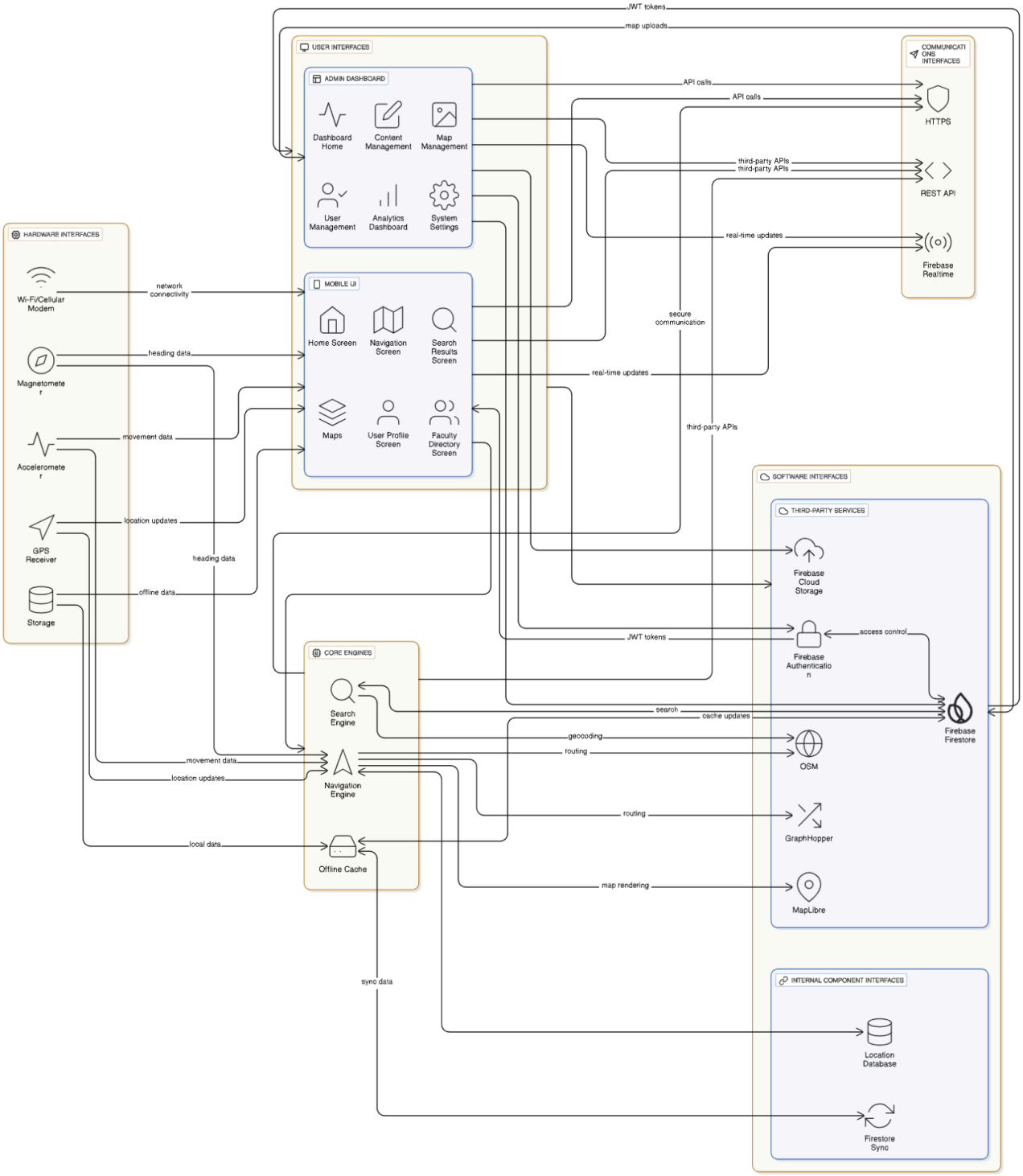
---

# Appendix B: Analysis Models

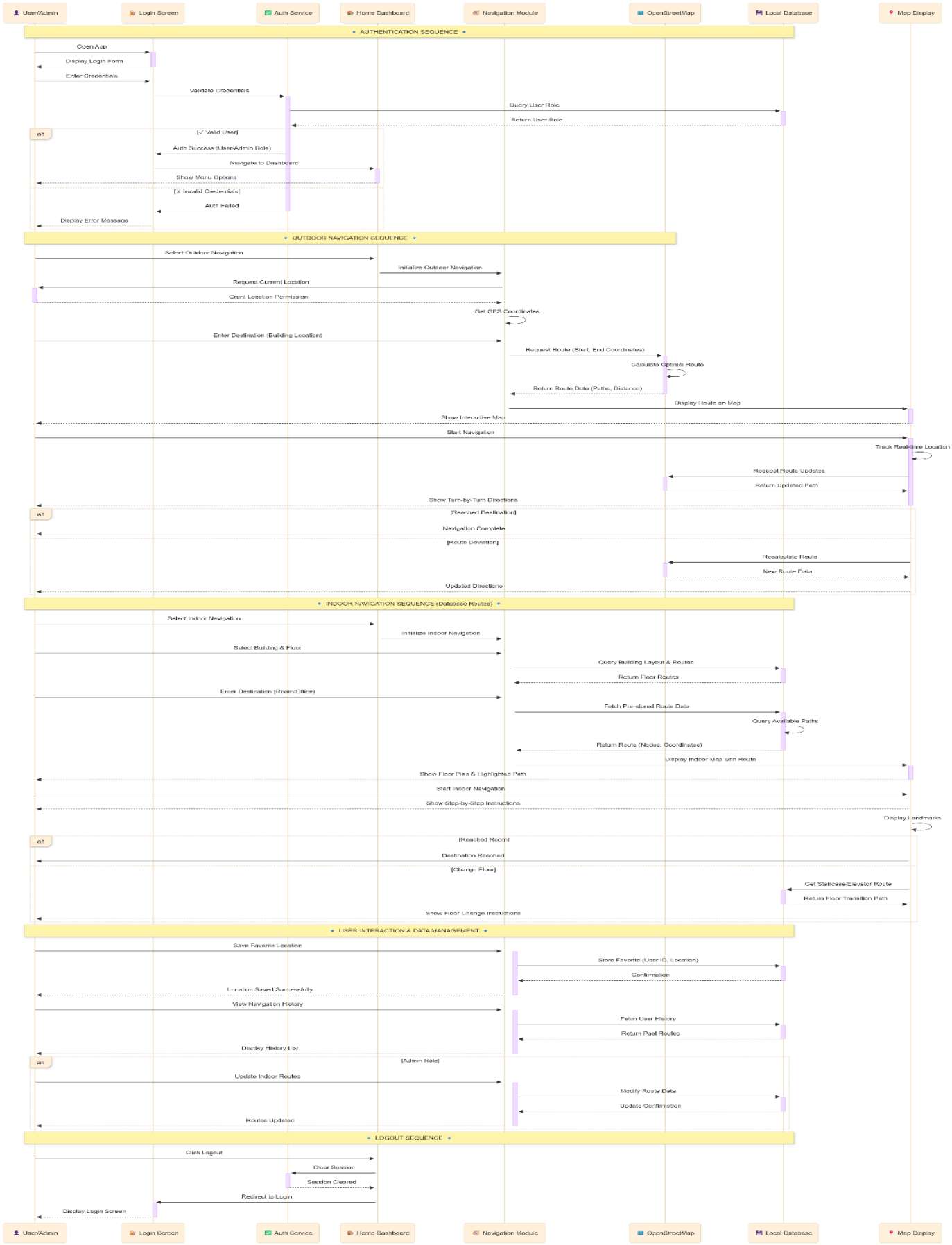
## B.1 Use Case Diagram



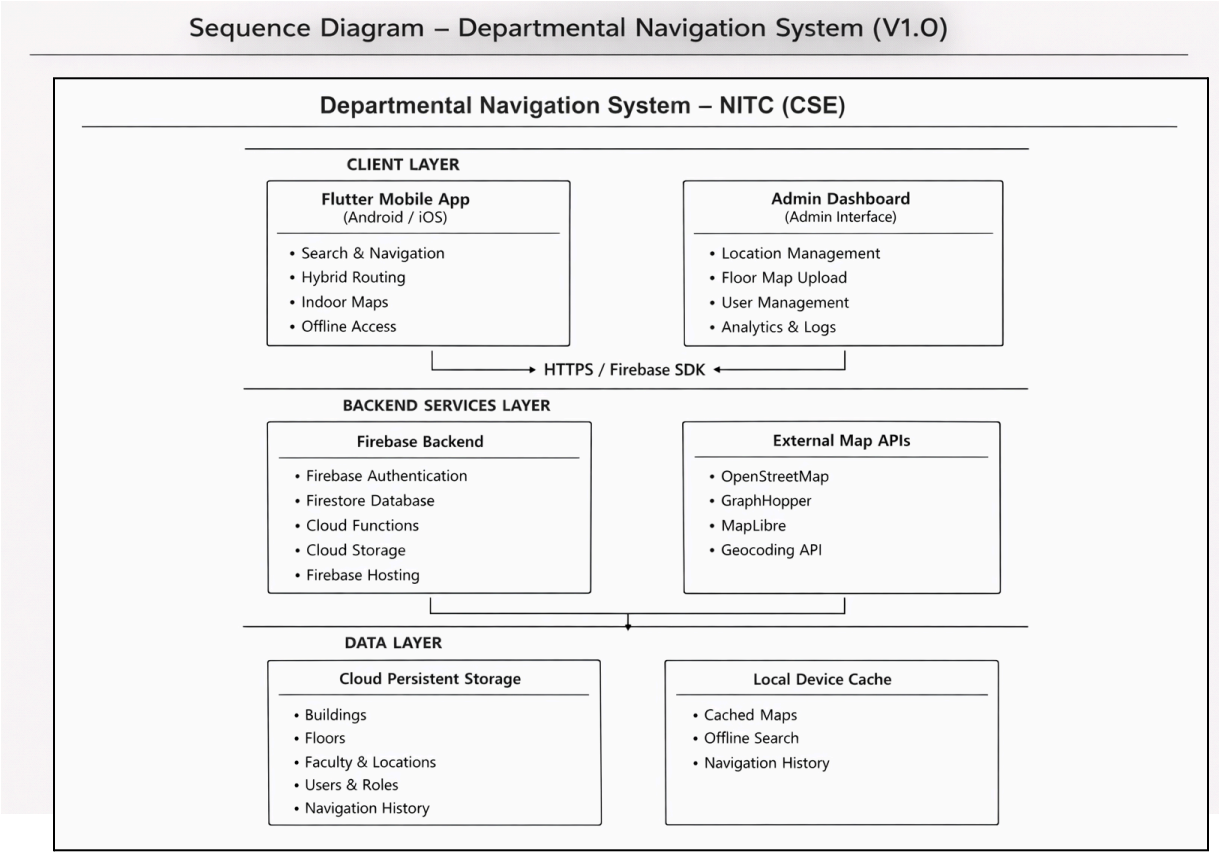

## B.2 Data Flow Diagram




## B.3 Sequence Diagram: Navigation Flow


User/Admin
Login Screen
Auth Service
Home Dashboard
Navigation Module
OpenStreetMap
Local Database
Map Display
AUTHENTICATION SEQUENCE
Open App
Display Login Form
Enter Credentials
Validate Credentials
Query User Role
Return User Role
alt
[✓ Valid User]
Auth Success (User/Admin Role)
Navigate to Dashboard
Show Menu Options
[✗ Invalid Credentials]
Auth Failed
Display Error Message
OUTDOOR NAVIGATION SEQUENCE
Select Outdoor Navigation
Initialize Outdoor Navigation
Request Current Location
Grant Location Permission
Get GPS Coordinates
Enter Destination (Building Location)
Request Route (Start, End Coordinates)
Calculate Optimal Route
Return Route Data (Paths, Distance)
Display Route on Map
Show Interactive Map
Start Navigation
Track Real-time Location
Request Route Updates
Return Updated Path
Show Turn-by-Turn Directions
alt
[Reached Destination]
Navigation Complete
[Route Deviation]
Recalculate Route
New Route Data
Updated Directions
INDOOR NAVIGATION SEQUENCE (Database Routes)
Select Indoor Navigation
Initialize Indoor Navigation
Select Building & Floor
Query Building Layout & Routes
Return Floor Routes
Enter Destination (Room/Office)
Fetch Pre-stored Route Data
Query Available Paths
Return Route (Nodes, Coordinates)
Display Indoor Map with Route
Show Floor Plan & Highlighted Path
Start Indoor Navigation
Show Step-by-Step Instructions
Display Landmarks
alt
[Reached Room]
Destination Reached
[Change Floor]
Get Staircase/Elevator Route
Return Floor Transition Path
Show Floor Change Instructions
USER INTERACTION & DATA MANAGEMENT
Save Favorite Location
Store Favorite (User ID, Location)
Confirmation
Location Saved Successfully
View Navigation History
Fetch User History
Return Past Routes
Display History List
alt
[Admin Role]
Update Indoor Routes
Modify Route Data
Update Confirmation
Routes Updated
LOGOUT SEQUENCE
Click Logout
Clear Session
Session Cleared
Redirect to Login
Display Login Screen
User/Admin
Login Screen
Auth Service
Home Dashboard
Navigation Module
OpenStreetMap
Local Database
Map Display

## B.4 System Architecture Diagram




# Appendix C: To Be Determined (TBD) List

| Item # | TBD Item | Description | Status | Priority |
|---|---|---|---|---|
| TBD-4 | AR Navigation | Augmented reality visualization for turn-by-turn directions | Pending | Low |
| TBD-6 | Multi-Department Support | Timeline and requirements for extending to other departments | Pending | Medium |
| TBD-9 | Accessibility Features | Specific accessibility requirements for differently-abled users | Pending | Medium |

---

## End of Software Requirements Specification

**Document Version:** 1.0
**Last Modified:** January 12, 2026